Prof. Dr. Erwin Mayer

Fakultät für Elektrotechnik
und Informationstechnik

Praktikum Verteilte Systeme

Versuch 2

RPC Programmierung



Was Sie in diesem Versuch lernen:

- Vertrautwerden mit einer RPC-basierten-Entwicklungs- und Laufzeitumgebung
- Einsatz einer IDL als Beschreibungssprache für verteilte Schnittstellen
- Kennenlernen eines RPC Stub- und Skeleton-Compilers
- Entwickeln eines RPC Cients und eines RPC-Servers
- Absprache von Schnittstellen im Team und verteilte Komponentenentwicklung
- Testen der RPC-Anwendung

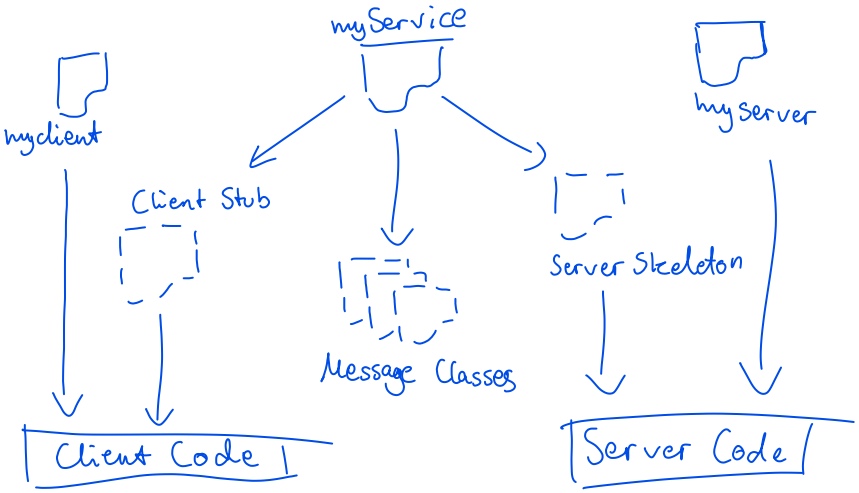
Voraussetzug zur Teilnahme: Pre-Lab Aufgaben komplett durchgeführt (mit Nachweis) !

15/04/26

1 PRE-LAB-Fragen

Name	Matrikelnummer	Semester
Nadine Bichler	192487	6

1. Skizzieren Sie, beginnend mit einem gRPC-proto-File, welche Teile der gRPC-Client-Server-Architektur aus dem proto generiert werden (gestrichelt) und welche selbst entwickelt werden müssen (nicht gestrichelt). Hinweis: Folie aus Vorlesung.



2. Definieren Sie über ProtoBuf v3 eine Schnittstelle für ein Serverobjekt, welches eine Mitgliederliste verwaltet (Listenelemente: Mitgl.-Nr., Name, Telefonnnummer. Operationen: Mitglied dazunehmen, Mitglied entfernen, Gesamte Liste abfragen) . (Hinweis: Wenn Sie keinen Parameter oder Returnwerte haben, müssen Sie dafür eine Empty-Message anlegen; es gibt hierfür google.protobuf.Empty)

```
message Member{
  int32 member_nr = 1;
  string name = 2;
  string phonenumber = 3;
}

message MemberList{
  repeated Member members = 1;
}

message MemberId{
  int32 member_nr = 1;
}

service MemberService{
  rpc GetMembers(google.protobuf.Empty) returns (MemberList);
  rpc AddMember(Member) returns (Member);
  rpc DeleteMember(MemberId) returns (google.protobuf.Empty);
```

3. Schauen Sie sich im Internet das gRPC-Proto-File https://github.com/grpc/grpc/blob/v1.35.0/examples/protos/route_guide.proto an. Machen Sie sich schlau, was die Anwendung macht. Weitere Erläuterungen zur Anwendung finden Sie unter https://grpc.io/docs/languages/java/basics/. Vervollständigen Sie unten auf dieser Seite den rpc-Typ, beschreiben Sie knapp, wofür jede der 4 Methoden dient und wofür das Streaming jeweils gebraucht wird.

1. rpc GetFeature (Point) returns (Feature) {}

   Was macht es? Obtains a feature at a given Position -> gibt einen Punkt zurück

2. rpc ListFeatures (Rectangle) returns (stream Feature) {}

   Was macht es?

   Obtains the Features available within the given Rectangle

   Wofür genau wird Streaming gebraucht?

   Results are streamed rather than returned at once (e.g. in a response message with a repeated field), as the rectangle may cover a large area and contain a huge number of features.

3. rpc RecordRoute(stream Point) returns (RouteSummary) {}

   Was macht es?

   Accepts a stream of Points on a route being traversed, returning a RouteSummary when traversal is completed.

   Wofür genau wird Streaming gebraucht?

   During the movement from a character the app is sending GPS-Points. The Server does not have to answer for every single Point but instead is collecting and at the end emitting

4. rpc RouteChat (stream RouteNote) returns (stream RouteNote) {}

   Was macht es?

   Accepts a stream of RouteNotes sent while a route is being traversed, while receiving other RouteNotes (e.g. from other users).

   Wofür genau wird Streaming gebraucht?

   real time interaction: communication is asynchronous, so client does not have to wait for answer and server can 'push' informations to cllient at all times so long as they are available